AI ASSISTED CODING

ASSIGNMENT 3.5

Name: M. Shashi

HtNo: 2303A51028

Question 1: Zero-Shot Prompting (Leap Year Check)

Write a zero-shot prompt to generate a Python function that checks whether a given year is a leap year.

Week2 -

Task:

• Record the AI-generated code.

• Test with years like 1900, 2000, 2024.

• Identify logical flaws or missing conditions.

```python
# Generate a python function that checks whether a given
def is_leap_year(year):
    try:
        year=int(year)
    except ValueError:
        return "Invalid Input"
    if (year % 4 == 0 and year % 100 != 0) or (year % 40
        return "Leap Year"
    else:
        return "Not a Leap Year"
year=input("Enter a year: ")
print(is_leap_year(year))s
```

PROBLEMS 1 OUTPUT TERMINAL PORTS ...

TERMINAL

```
varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-3_5.py
nter a year: 1900
Enter a year: 1900
varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-3_5.py
Enter a year: 1900
Not a Leap Year
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/v
arsh/AppData/Local/Programs/Python/Python313/python.exe c:/Users
/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-3_5.py
Enter a year: 2000
Leap Year
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/v
arsh/AppData/Local/Programs/Python/Python313/python.exe c:/Users
/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-3_5.py
Enter a year: 2004
Leap Year
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

The program does not check for invalid year values such as 0 or negative numbers. It only validates non-numeric input and assumes any integer year is valid. There is no explicit condition to restrict the year to a realistic range. Apart from these missing checks, the leap year logic itself has no flaws.

Question 2: One-Shot Prompting (GCD of Two Numbers)

Write a one-shot prompt with one example to generate a Python

function that finds the Greatest Common Divisor (GCD) of two numbers.

Example:

Input: 12, 18 → Output: 6

Task:

• Compare with a zero-shot solution.

• Analyze algorithm efficiency.

```python
a=12
b=18
display GCD is 6
'''
def compute_gcd(a,b):
    try:
        a=int(a)
        b=int(b)
    except ValueError:
        return "Invalid Input"
    while b:
        a,b=b,a%b
    return a
a=input("Enter first number: ")
b=input("Enter second number: ")
print("GCD is",compute_gcd(a,b))
```

PROBLEMS OUTPUT TERMINAL PORTS ...

TERMINAL

```
exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_a
signment-3_5.py
Enter first number: 12
Enter second number: 18
Enter second number: 18
GCD is 6
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

```python
#Generate python code that finds the greatest common divisor (GCD) of two numbers.
a=input("Enter first number: ")
b=input("Enter second number: ")
try:
    a=int(a)
    b=int(b)
except ValueError:
    print("Invalid Input")
    exit()
while b:
    a,b=b,a%b
print("GCD is",a)
```

PROBLEMS 1 OUTPUT TERMINAL PORTS

TERMINAL

```
Enter second number: 18
GCD is 6
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/P
/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-3_5.py
Enter first number: 12
Enter second number: 18
GCD is 6
```

Both solutions correctly compute the GCD using **the Euclidean algorithm,** which is an efficient and standard approach. The zero-shot solution is more straightforward and concise, while the one shot solution is better structured and reusable. In terms of efficiency, both have the same time complexity of O(log min(a, b)), which is very fast even for large numbers. There is no performance difference between them; the main difference is code organization and reusability, not logic or efficiency.

Question 3: Few-Shot Prompting (LCM Calculation)

Write a few-shot prompt with multiple examples to generate a Python

function that computes the Least Common Multiple (LCM).

Examples:

• Input: 4, 6 → Output: 12

• Input: 5, 10 → Output: 10

• Input: 7, 3 → Output: 21

Task:

• Examine how examples guide formula selection.

• Test edge cases.

```
a=4
b=6
display LCM is 12
a=5
b=10
display LCM is 10
a=0
b=0
display invalid input
a='abc'
b=5
display invalid input'''
def compute_lcm(a,b):
    try:
        a=int(a)
        b=int(b)
    except ValueError:
        return "Invalid Input"
    if a<=0 or b<=0:
        return "Invalid Input"
    def compute_gcd(x,y):
        while y:
            x,y=y,x%y
        return x
    gcd=compute_gcd(a,b)
    lcm=(a*b)//gcd
    return lcm
a=input("Enter first number: ")
b=input("Enter second number: ")
result=compute_lcm(a,b)
if result=="Invalid Input":
    print(result)
else:
    print("LCM is",result)
```

```
Enter first number: 0
Enter second number: 0
Invalid Input
PS C:\Users\varsh\OneDrive\ドキュメ
ograms/Python/Python313/python.exe c
b_assignment-3_5.py
Enter first number: 4
Enter second number: 6
LCM is 12
PS C:\Users\varsh\OneDrive\ドキュメ
ograms/Python/Python313/python.exe c
b_assignment-3_5.py
Enter first number: abc
Enter second number: 2
Invalid Input
```

Question 4: Zero-Shot Prompting (Binary to Decimal Conversion) Write a zero-shot prompt to generate a Python function that converts a binary number to decimal.

Task:

• Test with valid and invalid binary inputs.

• Identify missing validation logic.

```python
# generate a python function that converts a binary number to decimal numl
def binary_to_decimal(binary_str):
    try:
        binary_str=str(binary_str)
        decimal_value=0
        for index,digit in enumerate(reversed(binary_str)):
            if digit not in '01':
                return "Invalid Input"
            decimal_value+=int(digit)*(2**index)
        return decimal_value
    except ValueError:
        return "Invalid Input"
binary_str=input("Enter a binary number: ")
print(binary_to_decimal(binary_str))
```

```
ograms/Python/Python313/python.exe c:/Users/varsh/OneDrive/ドキュ
b_assignment-3_5.py
Enter a binary number: ml
Invalid Input
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/va
ograms/Python/Python313/python.exe c:/Users/varsh/OneDrive/ドキュ
b_assignment-3_5.py
Enter a binary number: 5.2
Invalid Input
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/va
ograms/Python/Python313/python.exe c:/Users/varsh/OneDrive/ドキュ
b_assignment-3_5.py
Enter a binary number: 11111
31
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

The code does not check for empty input, so an empty string would be treated as valid and return 0. It also does not handle leading/trailing spaces explicitly before processing. Negative binary values (like -101) are not considered and should be rejected clearly. Apart from these missing validations, the binary digit check (0 and 1) is correct.

Question 5: One-Shot Prompting (Decimal to Binary Conversion)

Write a one-shot prompt with an example to generate a Python function

that converts a decimal number to binary.

Example:

Input: 10 → Output: 1010

Task:

• Compare clarity with zero-shot output.

• Analyze handling of zero and negative numbers.

```python
10
display 1010
'''
def decimal_to_binary(n):
    try:
        n=int(n)
    except ValueError:
        return "Invalid Input"
    if n<0:
        return "Invalid Input"
    if n==0:
        return "0"
    binary_str=""
    while n>0:
        binary_str=str(n%2)+binary_str
        n=n//2
    return binary_str
n=input("Enter a decimal number: ")
print(decimal_to_binary(n))
```

PROBLEMS 1 OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh
ograms/Python/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメン
b_assignment-3_5.py
Enter a decimal number: 10
1010
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh
ograms/Python/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメン
b_assignment-3_5.py
Enter a decimal number: 0
0
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

```python
# generate o generate a Python code that converts a decimal number to binary
n=input("Enter a decimal number: ")
try:
    n=int(n)
except ValueError:
    print("Invalid Input")
    exit()
if n<0:
    print("Invalid Input")
elif n==0:
    print("0")
else:
    binary_str=""
    while n>0:
        binary_str=str(n%2)+binary_str
        n=n//2
    print(binary_str)
```

OBLEMS 1 OUTPUT TERMINAL PORTS

TERMINAL Python: I

```
/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-3_5.py
Enter a decimal number: 0
0
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Pro
/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-3_5.py
Enter a decimal number: -90
Invalid Input
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Pro
/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-3_5.py
Enter a decimal number: 2
10
```

Both versions correctly convert decimal to binary and handle zero and negative inputs. The zero-shot output (second code) is clearer for beginners because it is written step by step without a function, making the flow easy to follow. The function-based version is more organized and reusable but slightly less direct. Overall, the zero-shot solution is clearer in terms of readability, while the function version is better for reuse.

Question 6: Few-Shot Prompting (Harshad Number Check)

Write a few-shot prompt to generate a Python function that checks

whether a number is a Harshad (Niven) number.

Examples:

• Input: 18 → Output: Harshad Number

• Input: 21 → Output: Harshad Number

• Input: 19 → Output: Not a Harshad Number

Task:

• Test boundary conditions.

• Evaluate robustness

```
18
display harshad number
21
display harshad number
19
display not a harshad number
'''
def is_harshad_number(n):
    try:
        n=int(n)
    except ValueError:
        return "Invalid Input"
    if n<=0:
        return "Invalid Input"
    digit_sum=sum(int(digit) for digit in str(n))
    if n%digit_sum==0:
        return "harshad number"
    else:
        return "not a harshad number"
n=input("Enter a number: ")
print(is_harshad_number(n))
```

PROBLEMS 1 OUTPUT TERMINAL PORTS

TERMINAL

```
/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-3_5.py
Enter a number: 21
harshad number
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Prog
/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-3_5.py
Enter a number: 0
Invalid Input
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Prog
/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-3_5.py
Enter a number: -9
Invalid Input
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Robustness Evaluation The program is robust as it safely handles invalid inputs using try-except.It correctly rejects zero and negative numbers with clear error messages.Valid Harshad

numbers are accurately identified using digit-sum logic.Edge cases like non-numeric input are handled.